

2.680 Labs 07 and 08 - TSP

Distributed TSP and multi-machine TSP lab material.

Included MIT MOOS-IvP PDF sources:

- lab_class_07_distributed_tsp.pdf
- lab_class_08_multi_machine_tsp.pdf

Lab 7 - Distributed Traveling Salesman

2.680 Unmanned Marine Vehicle Autonomy, Sensing and Communications



March 5th, 2026

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

project-pavlab/labdocs/lab_class_07_distributed_tsp

1	Overview and Objectives	3
2	Preliminaries	3
3	Distributed Traveling Salesman - Preparing the Shoreside	5
3.1	Creating a Baseline Mission	6
3.2	Create a Script for Generating Visit Points	6
3.3	Create Shoreside pPointAssign MOOS App for Distributing Points	7
3.4	Sharing Visit Points to Your Vehicles	8
3.5	Visual Output of Visit Points	8
3.6	Issues to be Aware of - Gotchas	9
3.7	Assignment 1 (check off) - Point Assignment and Communication	10
4	Distributed Traveling Salesman - Preparing the Vehicle	11
4.1	Create a Vehicle pGenPath MOOS App for traversing points	11
4.2	Mission Constraints	11
4.3	Handling and Re-visiting Missed Points	12
4.4	Practical Suggestions	12
4.5	Assignment 2 (check off) - Confirm your mission runs	13
5	Suggestions for File Organization of Assignments	14
5.1	Requested File Structure	14
5.2	Due Date	14

1 Overview and Objectives

In today's lab we will continue our shift in focus to autonomy configurations involving multiple vehicles. Our lab exercise will involve a version of the Traveling Salesman Problem (TSP), in this case distributed over multiple vehicles. TSP is the problem of finding a path to visit a set of cities, with known distances between any two cities, in such a way as to visit every city at least once and to minimize the total distance traveled. We use the TSP problem to exercise our new skills in deploying multiple vehicles to handle a version of the TSP problem - the Distributed TSP problem. Given a set of points to visit, assign the points to a set of vehicles to visit those points. Since optimal TSP algorithms can be quite involved, we simplify things a bit and allow for non-optimal shortest tour distances while focusing on the mechanics of getting multiple cooperating vehicles running.

Gaining familiarity with this mode of operation will be essential for later labs and operation of vehicles on the water. In this lab:

- The `uTimerScript` utility will generate random sets of tour points on the Shoreside.
- A new MOOS app, `pPointAssign`, will be written by you and run on the Shoreside to distribute points to vehicles.
- Vehicles will be configured to receive visit point assignments from the Shoreside.
- A new MOOS app, `pGenPath`, will be written by you and run on the simulated robot to generate a tour of waypoints.
- Autonomy missions will be constructed to tour the points and return periodically for refueling.
- Resumed missions will finish any incomplete tour portion until the total tour is done.

2 Preliminaries

Make Sure You Have the Latest Updates

It is possible we will be making changes to the MOOS-IvP tree during the semester. Always make sure you have the latest code:

```
$ cd moos-ivp
$ git pull
Already up to date.
```

If you see a response similar to the one above, indicating that no changes were pulled in from the GitHub server, then you shouldn't need to re-build the software.

Otherwise you will need to rebuild:

```
$ ./build.sh
```

Do the same for the `moos-ivp-2680` tree.

Make Sure Key Executables are Built and In Your Path

This lab does assume that you have a working MOOS-IvP tree checked out and installed on your computer. To verify this make sure that the following executables are built and findable in your shell path:

```
$ which MOOSDB
/Users/you/moos-ivp/bin/MOOSDB
$ which pHelmIvP
/Users/you/moos-ivp/bin/pHelmIvP
```

If unsuccessful with the above, return to the steps in Lab 1:

http://oceanai.mit.edu/ivpman/labs/machine_setup

Where to Build and Store Lab Missions

As with previous labs, we will use your version of the moos-ivp-extend tree. In this tree, there is a missions folder:

```
$ cd moos-ivp-extend
$ ls
CMakeLists.txt  bin/          build.sh*    docs/        missions/    src/
README          build/        data/        lib/          scripts/
```

You will create a lab_07 folder/directory in the missions folder, typically with both a .moos and .bhv configuration file. See Section 5.1 for the full requested file structure.

Documentation Conventions

To help distinguish between MOOS variables, MOOS configuration parameters, and behavior configuration parameters, we will use the following conventions:

- MOOS variables are rendered in **green**, such as **IVPHELM.STATE**, as well as postings to the **MOOSDB**, such as **DEPLOY=true**.
- MOOS configuration parameters are rendered in **blue**, such as **AppTick=10** and **verbose=true**.
- Behavior parameters are rendered in **brown**, such as **priority=100** and **endflag=RETURN=true**.
- MOOS-IvP applications are rendered in **magenta**, such as **pShare**, or **pHelmIvP**.
- General GNU/Linux commands are represented in **dark purple**, such as **wget**, **mkdir**, or **cd**.

More MOOS / MOOS-IvP Resources

A few further resources are worth mentioning for following up this lab with your own exploration.

- The IvP Helm and Utilities documentation: <http://oceanai.mit.edu/ivpman>
- The moos-ivp.org website: <http://www.moos-ivp.org>

The Shoreside (Topside) / Vehicle Topology

The layout of interconnected MOOS communities used in this lab is depicted in the figure below. This layout will be used for the remainder of the course, including during operations on the river.

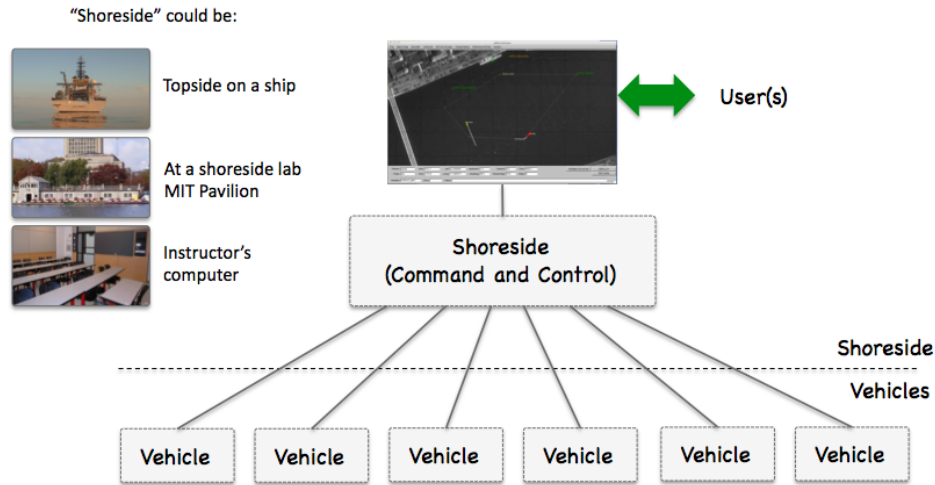


Figure 1: **Shoreside to Multi-Vehicle Topology:** A number of vehicles are deployed with each vehicle maintaining some level of connectivity to a shoreside command and control computer. Each node (vehicles and the shoreside) are comprised of a dedicated MOOS community. Modes and limits of communication may vary.

3 Distributed Traveling Salesman - Preparing the Shoreside

The Traveling Salesman Problem (TSP) is the problem of finding a path to visit a set of cities, with known distances between any two cities, in such a way as to visit every city at least once and to minimize the total distance traveled.

https://en.wikipedia.org/wiki/Travelling_salesman_problem

In this section we use the TSP problem to exercise our new skills in deploying multiple vehicles to handle a version of the TSP problem - the Distributed TSP problem. Given a set of points to visit, assign the points to a set of vehicles to visit those points. Since optimal TSP algorithms can be quite involved (NP-complete), we simplify things a bit and allow for non-optimal shortest tour distances while focusing on the mechanics of getting multiple cooperating vehicles running.

In this part we will:

- Create a `uTimerScript` script to generate 100 random points in a specified region.
- Create a `pPointAssign` MOOS App on the shoreside to send half the points to one vehicle and half to the other.
- Ensure our `pShare` communications is configured to share points out to the vehicles.
- Confirm it works by verifying that visit points are received on a pair of vehicles.

To create new apps, use the `GenMOOSAppAppCasting` utility. For example:

```
$ cd moos-ivp-extend/src
$ GenMOOSApp_AppCasting PointAssign p "Chris Jones"
pPointAssign generated
```

Recall that the `GenMOOSApp_AppCasting` script is in the `moos-ivp/scripts/` folder, and this needs to be in your shell path if it isn't already. Also don't forget that you'll need to add the new app to the `CMakeLists.txt` file in the `moos-ivp-extend/src` folder.

3.1 Creating a Baseline Mission

We will call the TSP mission `distributed_tsp`. A good place to start as a template is the last mission from the previous lab. If your file structure from Lab 6 followed the lab guidance, this should be `henry_gilda_auto_refuel`. We'll make a copy of that in our newly created `lab_07` folder.

```
$ cd moos-ivp-extend/missions
$ mkdir lab_07
$ cd lab_07
$ cp -rp ~/moos-ivp-extend/missions/lab_06/henry_gilda_auto_refuel distributed_tsp
```

The file structure for this lab is shown in Section 5.1.

3.2 Create a Script for Generating Visit Points

Configure a `uTimerScript` script running on the shoreside community, to generate a random sequence of points within a region of the operation area. The four corners of this region should be:

```
-25,-25
-25,-175
200,-25
200,-175
```

The script should generate 100 postings to the `MOOSDB` of the form:

```
VISIT_POINT = "firstpoint"
VISIT_POINT = "x=8, y=9, id=1"
...
VISIT_POINT = "x=-11, y=-9, id=100"
VISIT_POINT = "lastpoint"
```

The first posting in the script should be `VISIT_POINT="firstpoint"`. The last posting in the script should be `VISIT_POINT="lastpoint"`. These are the cues to recipients to indicate completeness of the set of points posted. The id values don't need to be perfectly in order, but should just be unique.

HINT: Consult the example script in the `uTimerScript` documentation in the example "A Script for Generating 100 Random Numbers" in the "Examples" section. The `uTimerScript` documentation is found:

<http://oceanai.mit.edu/ivpman/apps/uTimerScript>

Or, from the command line:

```
$ uTimerScript --web (or -w)
```

How do we know this is working? There are a few options:

- Check the output of the `uTimerScript` appcasting window.
- After quitting the mission, after the script has run, use the `allogrep` tool, on the alog file created in the shoreside log file, grepping on `VISIT_POINT`.
- After quitting the mission, launch `alogview` on the shoreside alog file and use the VarHist pull-down menu, selecting `VISIT_POINT`.

Note: In the last two options, we are relying on the contents of the alog file to confirm desired results. See the Section 3.6.2 below for an extra configuration of `pLogger` required in this mission. This is due to the fact that all our `VISIT_POINT` publications occur in a single burst.

3.3 Create Shoreside `pPointAssign` MOOS App for Distributing Points

Create a new `pPointAssign` MOOS application on the shoreside to send half the points to one vehicle and half to the other. The app should have the following capabilities:

- The app should subscribe to the `VISIT_POINT` output of the timer script described in Section 3.2 and produce output along the lines of:

```
VISIT_POINT_HENRY = "firstpoint"
VISIT_POINT_HENRY = "x=8, y=9, id=1"
...
VISIT_POINT_HENRY = "x=33, y=29, id=50"
VISIT_POINT_HENRY = "lastpoint"

VISIT_POINT_GILDA = "firstpoint"
VISIT_POINT_GILDA = "x=19, y=111, id=51"
...
VISIT_POINT_GILDA = "x=-11, y=-9, id=100"
VISIT_POINT_GILDA = "lastpoint"
```

Your app should follow the above format exactly. In a later lab we will want to combine one person's shoreside community, connected to another person's vehicle community. The above format will be important in this regard.

- The first and last postings made by `pPointAssign`, shared to each of the vehicles, should be `VISIT_POINT_VNAME="firstpoint"` and `VISIT_POINT_VNAME="lastpoint"`. These are the indicators usable on the vehicle that confirm that the complete set of visit points have been sent and received. The component `VNAME` above should be replaced by the vehicle name and there should be one such posting per vehicle.

- The app should have a configuration parameter, `vname`, which adds a known vehicle to the list of vehicles to which points will be distributed. In other words, the names `henry` and `gilda` shouldn't be hard-coded into the C++ code of `pPointAssign`, but should be parameters read in from a configuration file.
- Your app should support the assignment of points in one of two manners. In the first manner, points are assigned in an alternating fashion. The first point to the first vehicle, second point to the second vehicle, third point to the first vehicle and so on. In the second manner, points are assigned according to region, e.g., east-west. The app should be configurable to either way with a configuration parameter `assign_by_region` set to either true or false. You can see the difference in the two videos shown in Section 4.5.
- The east-west regions are simply the east and west halves of the region given earlier, with the four corners of $(-25, -25)$, $(-25, -175)$, $(200, -25)$, $(200, -175)$.
- **NOTE:** When distributing points by East/West region, it is *not* necessary to have an exactly even amount sent to each vehicle. Just split the region in half and distribute accordingly. Given the uniform distribution of the random points, they should be close to 50-50 on each side, but no guarantee.

3.4 Sharing Visit Points to Your Vehicles

You need to share `pPointAssign` postings out to their respective vehicles. For example you may want to share `VISIT_POINT_HENRY` out to vehicle `henry` with the arrival name `VISIT_POINT`. Remember that you should handle the share configuration with `uFldShoreBroker` now - not `pShare`.

3.5 Visual Output of Visit Points

It would be helpful to have your `pPointAssign` app also produce some visual output for `pMarineViewer`. This way you can verify that your `pPointAssign` app is (a) receiving points, and (b) grouping/handling them properly. Here is a suggested utility function for accomplishing this:

```
#include "XYPoint.h"    // Include this somewhere near the top of your file

...

void PointAssign::postViewPoint(double x, double y, string label, string color)
{
    XYPoint point(x, y);
    point.set_label(label);
    point.set_color("vertex", color); // yellow is handy on dark screen
    point.set_param("vertex_size", "4");

    string spec = point.get_spec();    // gets the string representation of a point
    Notify("VIEW_POINT", spec);
}
```

The label should be unique for each point, otherwise `pMarineViewer` will overwrite one point after another. A good label would just be the ID associated with the visit point. Another suggestion is to use a different color for each vehicle.

NOTE: The above code segment makes use of the `XYPoint` class, which is part of the `lib.geometry` library. To make use of this class (and others like it, such as `XYSegList`, `XPolygon` etc.) you will need to *link* against the `geometry` library. To link against this library, just add `geometry` to list of libraries that your app links to. This is found in the `CMakeLists.txt` file for your `pPointAssign` application.

```
$ cd moos-ivp-you/src/pPointAssign
$ emacs CMakeLists.txt (or whatever text editor you use)
```

Then add the line as below:

```
TARGET_LINK_LIBRARIES(pPointAssign
  ${MOOS_LIBRARIES}
  geometry      <--- Add this line
  mbutil
  m
  pthread)
```

Further note: In Linux the order of the libraries is important. And it's important to link against `geometry` first, before `mbutil` as above. For some reason the MacOS compiler is resilient to the order.

3.6 Issues to be Aware of - Gotchas

3.6.1 Handshaking

Recall that the MOOSDB only keeps track of the *most recent* value of a variable. When a new app connects and registers for mail, it will get only the latest value, not the entire history. Keep in mind that your `pPointAssign` app needs *all* the output of your timer script producing points. For this reason you will want to ensure that the script starts *after* `pPointAssign` has connected to the MOOSDB. Consider launching the timer script in a paused mode, where it gets automatically un-paused by the actions of `pPointAssign`. See the `pause_var` and `paused` configuration parameters for `uTimerScript`.

Likewise, eventually your `pPointAssign` app will send `VISIT_POINT` messages to your vehicles, to be handled in your `pGenPath` app. You will want to hold those outgoing messages from `pPointAssign`, until `pPointAssign` has received an indication that `pGenPath` app on the vehicle is ready to receive them.

3.6.2 Logging Initial Bursts of Data

The `pLogger` app is typically configured to "log everything" via the configuration parameter:

```
WildCardLogging = true
```

However, the way this is implemented in the logger is to monitor the list of variables known to the MOOSDB, published in the MOOS variable `DB_VAR.SUMMARY`. Once the logger has seen a new variable known to the MOOSDB, it will register for the variable. *However*, if the first time a variable is published to the MOOSDB, it is published in a burst of say 100 publications essentially at the same

time, then the logger may not log all publications. This is because, by the time `pLogger` notices the new variable on the `DB_VAR_SUMMARY` list, and then registers for subsequent publications, the initial burst has finished and the logger was not able to register for them in time.

The way to avert this problem is to configure the logger to register for such variables at the outset. We can still configure wildcard logging, but we can also tell the logger to register for certain variables without waiting for them to be seen on the `DB_VAR_SUMMARY` list. This can be done with the following configuration line:

```
Log = VISIT_POINT @ 0 nosync
```

The logger can be configured with as many of this lines as desired.

3.7 Assignment 1 (check off) - Point Assignment and Communication

Demonstrate the successful implementation of your `pPointAssign` app, and the configuration of communications to your vehicles.

- Demonstrate that your `pPointAssign` app is receiving and processing (grouping) all points by rendering the groups in different colors in `pMarineViewer`. An example would be the image on the cover page of this lab.
- Demonstrate that your `pPointAssign` app is sending a proper group of points to one vehicle and the other part to the other vehicle. This is simply to confirm that the share configuration is properly configured.

To demonstrate things are working, you have two options. The first is the method traditionally used, before `RealmCasting` was available. Still works, but probably harder and more cumbersome:

- First method of demonstration: open two terminal windows, one for each vehicle, running `uXMS` with the `--history` option scoping on `VISIT_POINT`.
- Second method of demonstration: Add `pRealm` to the set of apps running on the shoreside and vehicle communities. In the vehicles, in the `pRealm` config block, add the configuration line: `hist_var = VISIT_POINT`. When the mission is launched, in `pMarineViewer`, you can toggle between `AppCasting` and `RealmCasting` by using the 'a' key. Then you can select the vehicle and the history variable.

4 Distributed Traveling Salesman - Preparing the Vehicle

The vehicles in this lab will be sent a set of visit points from the Shoreside, as described in the previous section. On the vehicle a new MOOS app is needed to ingest those points, wait until all have been received, and generate a path for visiting all points. That is the focus of this part of the lab.

In this part we will:

- Create `pGenPath` MOOS App to be run on the vehicle side. It should subscribe for the points coming from `pPointAssign` on the shoreside, generate a sequence of waypoints, and then publish them to the helm as an update to a waypoint behavior.
- Configure an autonomy mission on the vehicle to traverse the points while periodically returning to home. We will simulate a vehicle with limited energy for surveying, by mandating that it return home for refueling after some amount of time being under way.

4.1 Create a Vehicle `pGenPath` MOOS App for traversing points

Create a new `pGenPath` MOOS module for running on the vehicle, that takes as input a series of `VISIT_POINT` messages from the shoreside, and generates a waypoint list suitable for consumption by the waypoint behavior.

In your behavior configuration, use a Waypoint behavior with an `updates` parameter specifying the MOOS variable used for incoming dynamic parameter configuration. In this case, the primary incoming configuration will be the list of waypoints from your `pGenPath` module. To make things a bit easier, *your shortest path algorithm may simply be a greedy shortest path* with an imaginary virtual initial point being the vehicle's present position.

In your `pGenPath` module you can use the following pseudo code as guidance for how to build a proper list of waypoints recognized by the waypoint behavior:

```
XYSegList my_seglist;
my_seglist.add_vertex(3, 8);
my_seglist.add_vertex(43, 99);
my_seglist.add_vertex(44, -111);

string update_str = "points = ";
update_str      += my_seglist.get_spec();
Notify("UPDATES_VAR", update_str); // UPDATES_VAR depends on your config
```

If your application uses the `XYSegList` class (recommended), your app must link against the *geometry* library. You will need to edit the `CMakeLists.txt` file in your `pGenPath` application folder. In this file, simply add `geometry` in the `TARGET_LINK_LIBRARIES` block. Similar to what you probably also needed to do for the `pPointAssign` app in the previous part of the lab.

4.2 Mission Constraints

- *Maximum vehicle speed:* The vehicle may go no faster than 1.2 meters/sec.

- *Tour Duration*: The vehicle may be underway no longer than 10 minutes (600 seconds) before re-fueling will be required. To simplify things, you need not include the time required to return to home in this time limit.
- *Refuel Duration*: Once the vehicle has returned home, it must wait at least 60 seconds before resuming its mission.

4.3 Handling and Re-visiting Missed Points

You may find that the tour generated by a greedy algorithm has turns that the vehicle simply cannot make quickly enough, causing the vehicle to "miss" a point on the tour. You should use the `slip_radius` parameter (in the Waypoint behavior) with a high enough number, e.g., 15, such that the vehicle does *not* come back around for missed points. For our lab, a visit to a point is satisfied if the vehicle comes within three meters of the point.

In the *next* lab, we will re-design this mission such that, upon completion of the tour, the `pGenPath` app makes a new tour of all points missed on the previous tour. The `pGenPath` app can keep track of visited points by simply noting the present vehicle position on each iteration, and comparing it to known points in the tour. Upon each completion of a tour, or modified tour, the vehicle should return home for re-fueling regardless of how long it has been deployed since the previous re-fueling.

For now, we won't worry about "missed" points or re-visiting points on a second tour. We mention this now in case you want to keep in mind this upcoming challenge in building your `pGenPath` application.

4.4 Practical Suggestions

Make `pGenPath` an `AppCastingMOOSApp`

In your `pGenPath` module, keep records of various state values, e.g., number of points received, visited, unvisited etc. Below is the appcast output of my own implementation. Store and report whatever helps you debug your application.

```
=====
pGenPath henry                                0/0(6011)
=====
Visit Radius:          3
Total Points Received: 55
Invalid Points Received: 0
First Point Received:  true
Last Point Received:   true
NAV_X/Y Received:     true

Tour Status
-----
Points Visited:        55
Points Unvisited:      0
```

Make pPointAssign an AppCastingMOOSApp

For the same reasons as above, you may find good appcasting output for pPointAssign will help you debug.

4.5 Assignment 2 (check off) - Confirm your mission runs

When your pPointAssign application is configured to assign points in an alternating manner as they come in, your mission may look something similar to the video posted below. Note that in this mission the vehicle returns to visit points missed on the first tour. You don't need to be concerned with this aspect of the mission until the next stage, in the next lab.

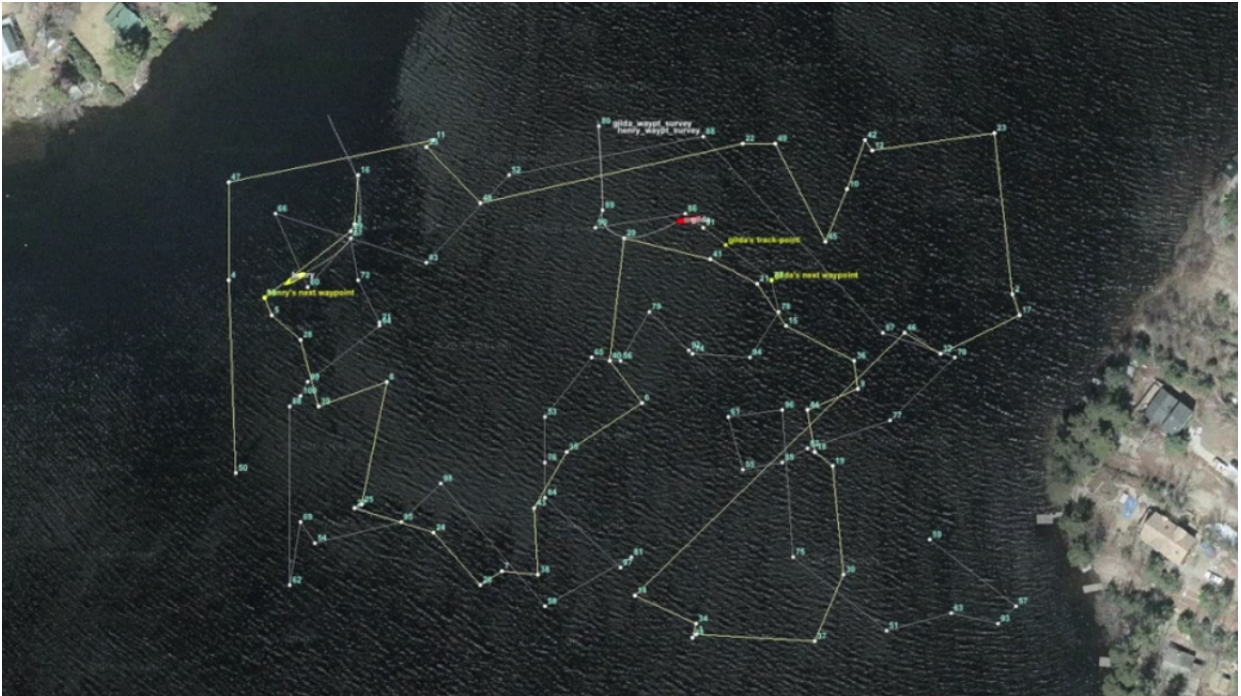


Figure 2: In the First Distributed TSP mission two vehicles are assigned randomly chosen points in the region to visit on a first-come basis. Therefore their paths overlap and the mission is somewhat inefficient, but at least the same amount of points are assigned to each vehicle.

When pPointAssign is configured to assign points by region, your mission may look something like the video posted below. Note that in this mission the vehicle returns to visit points missed on the first tour. You don't need to be concerned with this aspect of the mission until the next lab.



Figure 3: In the Distributed TSP Two mission two vehicles are assigned randomly chosen points in the region based on the section of the region. Therefore their paths do not overlap and the mission is somewhat more efficient.

5 Suggestions for File Organization of Assignments

5.1 Requested File Structure

You are encouraged to continue to upload all mission files. Here is the requested file structure:

```
moos-ivp-extend/
  missions/
    lab_07/
      distributed_tsp/
  src/
    pPointAssign/
    pGenPath/
```

5.2 Due Date

Assignment 1 of this lab should be finished by the beginning of lab, Thursday, March 12th, 2026.

Assignment 2 of this lab is due by the end of lab, Thursday March 12th, 2026.

Lab 08 - Multi-Machine TSP with Re-planning

2.680 Unmanned Marine Vehicle Autonomy, Sensing and Communications



March 12th, 2026

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1	Overview and Objectives	3
2	Preliminaries	3
3	Experimenting with Distributed Simulation	5
3.1	Lab IP Address Requirements for Distributed Simulations	6
3.2	Local and Publicly Available IP Addresses	6
4	Run the Double-Vehicle Mission On Multiple Machines	8
4.1	Obtain a Copy of the Lab 8 Baseline mission Configuration	8
4.2	Confirm the Baseline Mission is Runnable on a Single Machine	9
4.3	Work With a Lab Partner to Launch the Baseline Mission	10
4.4	Assignment 1 (check off) - Demonstrate Double Loiter on Two Machines	12
5	Add Re-planning to the Distributed TSP Mission	13
5.1	Mission Folder Preparation	13
5.2	Re-visiting Missed Points	13
5.3	Requirements for this Exercise	13
5.4	Assignment 2 (check off) - Distributed TSP Two	14
6	Distributed TSP Mission on Multiple Machines	15
6.1	Preparing Your Mission for Launching on Multiple Machines	15
6.2	Assignment 3 (check off) - Distributed TSP Mission on Multiple Machines	15
7	Instructions for File Organization of Assignments	16
7.1	Requested File Structure	16
7.2	Due Date	16

1 Overview and Objectives

The focus on today's lab is (a) to begin working with multi-vehicle missions with separate physical machines (for now simulated) vehicles and a separate remote Shoreside machine, and (b) to extend our distributed traveling salesman solution to include re-planning when the robot mission misses points on the first tour. In this lab, we will:

- Run a simple baseline double-vehicle mission over multiple machines,
- Extend the Lab 07 Distributed TSP mission with re-planning,
- Run the Distributed TSP mission over multiple machines.

The goal of the first and third items above is to begin migrating our lab work-flow to make use of an in-lab simulation environment where individual (or group) autonomy configurations are demonstrated. In short, the goal is to replace the shoreside MOOS community you have been running on your laptops with a single classroom shoreside community to which you connect your simulated vehicle. This is much closer to the setup that will be used during robot operation on the Charles river.

The goal of the second (re-planning) item above is to gain more familiarity with behavior configurations and MOOS app development by implementing a further capability on your Lab 7 TSP assignment.

2 Preliminaries

The MOOS-IvP-2680 Tree

Starting with Lab 6 and continuing with this lab, we use the additional [moos-ivp-2680](#) tree. In total we work with three trees:

- [moos-ivp](#): Public autonomy codebase (provided to you)
- [moos-ivp-2680](#): 2.680 autonomy codebase and baseline missions (provided to you)
- [moos-ivp-extend](#): Student codebase and lab solution missions (created by you)

If you haven't done so already, download this tree onto your computer in the same folder, side-by-side with [moos-ivp](#) and [moos-ivp-extend](#). To download:

```
$ git clone git@github.com:mit2680/moos-ivp-2680.git
```

** If you already have this repo, please perform a `git pull`, to ensure you have any recent updates.
**

The [moos-ivp-2680](#) tree contains both MOOS apps and MOOS missions useful to 2.680. In today's lab, content from the [moos-ivp-2680/missions](#) folder will be needed. We don't yet need any apps, but if you want to add [moos-ivp-2680/bin](#) to your shell path in your `.bashrc` file, this is a good move you will need for later.

Make Sure Your Machine Hostname is Set

Make sure your machine hostname is set to a meaningful value. To check your hostname, run the following from the command line:

```
$ hostname
```

If you're using one of the lab MacBooks (or your own Mac), you may be surprised to see it return a number instead of the lab assigned machine name. If this is the case, set your hostname to match the name assigned on your machine label using the following command:

```
$ sudo scutil --set HostName <your-machine-name>
```

Make Sure You Have the Latest Updates

Always make sure you have the latest code:

```
$ cd moos-ivp
$ git pull
```

And rebuild if necessary:

```
$ ./build-moos.sh
$ ./build-ivp.sh
```

Make Sure Key Executables are Built and In Your Path

This lab does assume that you have a working MOOS-IvP tree checked out and installed on your computer. To verify this make sure that the following executables are built and findable in your shell path:

```
$ which MOOSDB
/Users/you/moos-ivp/bin/MOOSDB
$ which pHelmIvP
/Users/you/moos-ivp/bin/pHelmIvP
```

If unsuccessful with the above, return to the steps in Lab 1:

http://oceanai.mit.edu/ivpman/labs/machine_setup

Where to Build and Store Lab Missions

As with previous labs, we will use your version of the moos-ivp-extend tree. In this tree, there is a missions folder:

```
$ cd moos-ivp-extend
$ ls
CMakeLists.txt  bin/          build.sh*     docs/         missions/     src/
README          build/        data/         lib/          scripts/
```

For each distinct assignment in this lab, there should be a corresponding sub-directory in a `lab.08` sub-directory of the `missions` folder, typically with both a `.moos` and `.bhv` configuration file. See Section 7.1 for the full requested file structure.

Documentation Conventions

To help distinguish between MOOS variables, MOOS configuration parameters, and behavior configuration parameters, we will use the following conventions:

- MOOS variables are rendered in **green**, such as `IVPHELM.STATE`, as well as postings to the `MOOSDB`, such as `DEPLOY=true`.
- MOOS configuration parameters are rendered in **blue**, such as `AppTick=10` and `verbose=true`.
- Behavior parameters are rendered in **brown**, such as `priority=100` and `endflag=RETURN=true`.
- MOOS-IvP applications are rendered in **magenta**, such as `pShare`, or `pHelmIvP`.
- General GNU/Linux commands are represented in **dark purple**, such as `wget`, `mkdir`, or `cd`.

More MOOS-IvP Resources

- See the slides from class which give a bit more background into marine autonomy, the IvP Helm, and uField Toolbox.

http://oceanai.mit.edu/2.680/docs/2.680-06-multiple_vehicles_2026.pdf

http://oceanai.mit.edu/2.680/docs/2.680-07-ivphelm_behaviors_2026.pdf

http://oceanai.mit.edu/2.680/docs/2.680-08-multiple_vehicles_2026.pdf

http://oceanai.mit.edu/2.680/docs/2.680-09-behaviors_deepdive_2026.pdf

- The uFldNodeBroker documentation

<http://oceanai.mit.edu/ivpman/apps/uFldNodeBroker>

- The uFldShoreBroker documentation

<http://oceanai.mit.edu/ivpman/apps/uFldShoreBroker>

3 Experimenting with Distributed Simulation

The first exercise in today's lab involves simulation distributed over more than one machine. Note that in labs 6 and 7, the mission configuration structure began to take the form of (a) single shoreside MOOS community running `pMarineViewer`, and (b) some number of vehicle communities. All of these communities were still running on a single machine, e.g., your laptop. When our labs begin to use robots the Charles River, this of course will change, but the logical topology will be the same.

As a step in this direction, in today's lab we experiment with simulations distributed over multiple machines.

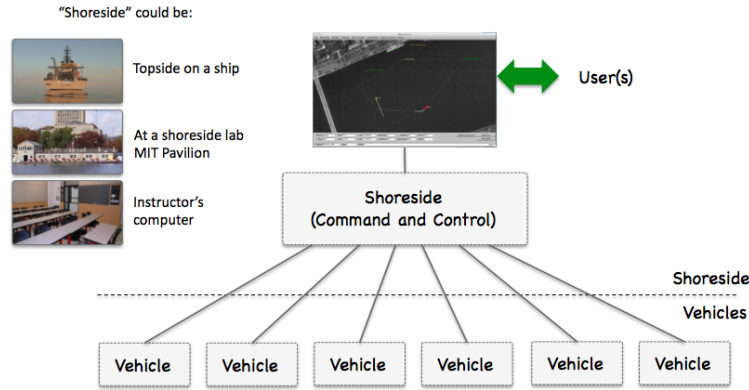


Figure 1: **Shoreside to Multi-Vehicle Topology:** A number of vehicles are deployed with each vehicle maintaining some level of connectivity to a shoreside command and control computer. Each node (vehicles and the shoreside) are comprised of a dedicated MOOS community. Modes and limits of communication may vary.

3.1 Lab IP Address Requirements for Distributed Simulations

The multi-vehicle topology described above requires that (a) the shoreside IP address is known to each of the vehicles, and (b) the shoreside also knows the IP address for all vehicles. When we launch our shoreside community, we will declare the shoreside IP address:

```
$ ./launch_shoreside.sh --ip=18.18.23.149
```

And when a remote vehicle is launched, we will also declare the vehicle's IP address, but also indicate where we expect to find the shoreside:

```
$ ./launch_vehicle.sh --ip=72.74.58.121 --shore=18.18.23.149 --sim
```

The rest is taken care of automatically by the `uFldShoreBroker` app running on the shoreside, and the `uFldNodeBroker` app running in the vehicle community.

Our requirement is that each side can actually reach each other at the specified IP address. Just because your machine is connected to the Internet and can browse the web and so on, doesn't mean your machine is reachable and ready to participate in our multi-vehicle environment.

3.2 Local and Publicly Available IP Addresses

The IP address space (Internet Protocol Version 4) has 2^{32} or roughly 4.3 billion possible unique IP addresses. This may have seemed like a sufficiently large number in the early Internet years, but in practice it is nowhere near enough today. There are at least a couple ways of dealing with this. One way is IPv6, or Internet Protocol Version 6 (Version 5 was apparently just an experimental protocol). IPv6 has 128 bits instead of the 32 bits in IPv4, so it should accommodate humanity for some time.

Another way is to conserve IPv4 addresses, by "sharing" a single IPv4 address among local users. Perhaps the best case example is a home router. There may be a dozen or many more devices

connected to a router in a house, including laptops and phones, but all these devices share a single IPv4 address.

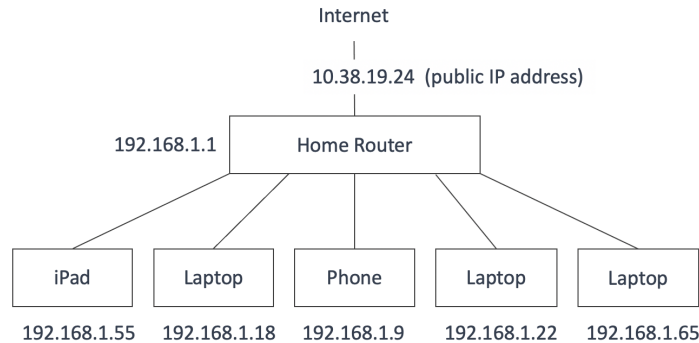


Figure 2: A typical home network with a local router. Each device has a local (192.168.x.x) IP address, with the home as a whole using a single IPv4 public IP address.

It would be hard to connect directly to any single home device from the outside world, via say `ssh`. If you were to try:

```
$ ssh 10.38.19.24
```

The `ssh` interaction could be meant for any one of the computers on the home network, how would the router know which computer the outside user was trying to `ssh` into? The `ssh` protocol uses port 22, and the router could be configured to re-direct incoming traffic over port 22 to a particular computer. This is called *port forwarding*. In our case in 2.680, we are concerned with traffic over `pShare`. Similar to `ssh`, we could configure our home router to forward incoming traffic over port 9200 (a common port we choose to use in our missions), to a particular machine.

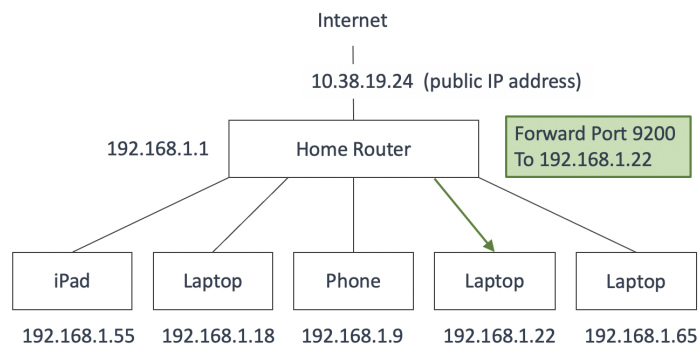


Figure 3: Port forwarding can be configured on a home router to re-direct incoming traffic on a particular port to a particular machine.

For most use-cases in 2.680, we will all be working on the same 192.168.* (or similar) network in the classroom. It is possible however to run the multi-machine simulation across machines from your home to your lab partner at home, if you would like to take a crack at configuring your home

routers with port forwarding. If you do end up trying this, let us know if you succeed!

4 Run the Double-Vehicle Mission On Multiple Machines

In this part we will:

- Obtain a copy of the lab 08 baseline mission configuration.
- Confirm that it is runnable on your own machine.
- Work with a lab partner to test it over two machines.

4.1 Obtain a Copy of the Lab 8 Baseline mission Configuration

The first step is to copy the baseline example mission from the `moos-ivp-2680` tree and copy it into your own `moos-ivp-extend` tree.

```
$ mkdir moos-ivp-extend/missions/lab_08
$ cp -rp moos-ivp-2680/missions/lab_08_baseline moos-ivp-extend/missions/lab_08
```

It should look like:

```
moos-ivp-extend/
missions/
  lab_08/
    lab_08_baseline/
```

The `lab_08_baseline` mission files should like:

```
$ ls missions/lab_08/lab_08_baseline
clean.sh*      launch_vehicle.sh*  meta_vehicle.moos
launch.sh*     meta_shoreside.moos plugs.moos
launch_shoreside.sh* meta_vehicle.bhv    init_field.sh
```

The most important part to use from this baseline is the `launch*.sh` files. You can use the `*.moos` and `*.bhv` files from your Lab 07 mission folder.

Note: this mission file uses the condensed style of plugs where all plugs common to the vehicle and shoreside are in the single file, `plugs.moos`. Each block is separated by a tag. The inclusion of plugs inside the meta file then uses the tag, e.g., `#include plugs.moos <pShare>`.

As with previous missions, the actual `targ_*` mission files are auto-generated upon executing the launch scripts. Here there are *three* launch scripts - one (`launch_shoreside.sh`) if you are launching the shoreside community on your machine and serving others, and the other (`launch_vehicle.sh`) if you are launching a vehicle community and connecting to a remote shoreside community. The `launch.sh` script is used when you want to test the mission running all three communities on your machine. All three scripts support the `-h` switch and both support the `--just_make` (or simply `-j`) switch if you want to just build the target files without actually launching. Below is an example (with username `mikerb`):


```
$ cd lab_08/lab_08_baseline
$ ./launch_shoreside.sh --just_make
$ ./launch_vehicle.sh --just_make --sim
$ ls
clean.sh*          meta_shoreside.moos  targ_mikerb.bhv
launch.sh*         meta_vehicle.bhv     targ_mikerb.moos
launch_shoreside.sh* meta_vehicle.moos    targ_shoreside.moos
launch_vehicle.sh*  plugs.moos
```

Note the three target files newly created. And the target files should not be under version control. You should have a `.gitignore` file in your `moos-ivp-extend` tree with a line `*targ_*`, to ensure you don't accidentally put these files under version control. Running the `clean.sh` script should remove both the `targ_*` files and the log files. I use the following alias in my `.bashrc` file. Feel free to add it if you like it:

```
alias lll='./clean.sh; ktm; ./launch.sh'
```

Then you can launch a mission, using time warp 20 for example, with:

```
$ lll 20
```

4.2 Confirm the Baseline Mission is Runnable on a Single Machine

The goal of this step is to launch two communities with the two specialized shell scripts provided in the `lab_08_baseline` mission. Although the intention is that typically you would launch just one or the other, launching both on the same machine should work just fine. To do this, first launch the shoreside community with a time warp of ten:

```
$ ./launch_shoreside.sh 10 -v
```

With the `-v` or `--verbose` option, the launch scripts will present to you output like that shown below, before continuing to launch. This allows you confirm the key launch parameters, as a form of debugging. Once you have things working, you can drop the `-v` command line argument.

```

$ ./launch_shoreside.sh 10 --verbose
=====
launch_shoreside.sh SUMMARY
=====
launch_shoreside.sh
CMD_ARGS =      [-v 10 ]
TIME_WARP =      [10]
JUST_MAKE =      [no]
AUTO_LAUNCHED = [no]
-----
IP_ADDR =        [10.200.14.243]
MOOS_PORT =      [9000]
PSHARE_PORT =    [9200]
LAUNCH_GUI =     [yes]
MMOD =           []
-----
V NAMES =        []
-----
Hit any key to continue launch

```

This allows you to verify the values of the important settings before continuing. To disable this prompt and just launch immediately, launch instead without the `--verbose`, `-v` flag.

Note, when launching on a separate machine, you may need to provide the public IP address explicitly:

```
$ ./launch_shoreside.sh --ip=1.2.3.4 10 (time warp 10)
```

Then launch a vehicle community with a matching time warp:

```
$ ./launch_vehicle.sh 10 --sim (or -s)
```

In launching the vehicle community as above with no command-line arguments other than the time warp, defaults are used for the vehicle name (`username`), MOOSDB ServerPort (9001), and `pShare` listen port (9201). In launching a second or more vehicles, these arguments need to be provided with different values on the command line. For example, try launching a second vehicle with:

```
$ ./launch_vehicle.sh --vname=henry --mport=9002 --pshare=9202 --sim 10
```

It may seem strange that there are multiple `pMarineViewer` windows launched. But this is more sensible when these two communities are being launched on separate machines, as intended normally. The *shoreside* `pMarineViewer` will render all vehicles. The *vehicle* `pMarineViewer` will render just what is known locally.

4.3 Work With a Lab Partner to Launch the Baseline Mission

In the next step, find a lab partner to work with launching the baseline mission. Take turns playing the vehicle role and the shoreside role. If you want to work with multiple partners to launch several vehicles connected to a single shoreside community, that's great.

As before, the first step is to launch the shoreside community. Pick a desired time warp and inform those connecting with a vehicle of this value:

```
$ ./launch_shoreside.sh --ip=<your-ip-address> <time-warp>
```

OR if working on the same subnet, simply:

```
$ ./launch_shoreside.sh <time-warp>
```

Noting the shoreside IP address: To next launch a vehicle community on another machine, the person launching the vehicle community needs to know the IP address of the shoreside community. This can be found many ways but perhaps the easiest is to just look at the top of the [pMarineViewer](#) window. Once this is known, the vehicle community is launched with:

```
$ ./launch_vehicle.sh --ip=<your-ip-address> --shore=<ip-address-for-shore> --vname=<vname>  
--mport=<port> --pshare=<port> <time-warp> --sim
```

If you are only launching one vehicle per machine, the `--vname`, `--mport`, and `--pshare` arguments may be omitted. The default values will suffice. They only need to be specified to make sure that when launching more than one vehicle on a single machine that the vehicle name, [MOOSDB](#) port and [pShare](#) input ports are unique for each machine.

Regarding the network IP address on the two machines, note that it's important that either:

- Both machines are on the same local network. This is the case when the IP address is assigned by a router perhaps in the room. In this case the IP address may begin with one of a couple special numbers, `192.168.x.x`, or `10.1.x.x`.
- Both machines are on the global network, with an IP address known to the rest of the Internet.

One way to test your network setup between the two machines is if both of you are able to ping each other using the [ping](#) command. This utility is found at `/sbin/ping` on most machines. Test the [ping](#) in both directions.

If you've never used [ping](#) before, you can try pinging the class server. A successful connection looks like below. (hit ctrl-c to quit)

```
$ ping 18.18.38.22  
64 bytes from 18.18.38.22: icmp_seq=0 ttl=49 time=62.625 ms  
64 bytes from 18.18.38.22: icmp_seq=1 ttl=49 time=101.966 ms  
64 bytes from 18.18.38.22: icmp_seq=2 ttl=49 time=131.908 ms
```

An unsuccessful ping would look like:

```
$ ping 18.18.38.22
PING 18.18.38.22 (18.18.38.22): 56 data bytes
Request timeout for icmp_seq 0
Request timeout for icmp_seq 1
Request timeout for icmp_seq 2
Request timeout for icmp_seq 3
```

If you can't [ping](#) or connect to your lab partner's machine, you can try disabling the firewall. See a lab TA if you need help on this.

4.4 Assignment 1 (check off) - Demonstrate Double Loiter on Two Machines

To demonstrate this assignment, you can either:

- Work with a partner, one acting as a shoreside, the other partner launching a vehicle, or preferably two vehicles. Or
- Work with one of the TAs, with the TA providing the second community, either shoreside or vehicle.

5 Add Re-planning to the Distributed TSP Mission

The next exercise in this lab is to augment the lab 7 Distributed TSP mission with a re-planning component.

5.1 Mission Folder Preparation

We'll make a new mission folder in our `missions/lab_08` folder which, for starters, will be just a direct copy from your Lab 7 mission folder:

```
$ cd moos-ivp-extend/missions/lab_08
$ cp -rp moos-ivp-extend/missions/lab_07/distributed_tsp distributed_tsp_replan
```

Now things should look like:

```
moos-ivp-extend/
missions/
  lab_08/
    lab_08_baseline/      // Assignment 1 - check off
    distributed_tsp_replan/ // For now a copy of lab 7 distributed_tsp mission
```

5.2 Re-visiting Missed Points

Note that when the vehicle makes sharp turns, it often "achieves" its next waypoint even though it may miss going directly over the point by a wide margin. This is due to generous settings of the capture radius and slip radius parameters in the waypoint behavior. (For example, if the `slip_radius` is 20 meters, and the `capture_radius` is 10 meters, the vessel may "achieve" the waypoint, but may never come closer than say 18 meters.)

In this exercise, we will define a separate notion of what it means to have successfully visited a waypoint. We augment the `pGenPath` application to keep track of which points were visited by its own observation and criteria for achieving a visit. Once the vehicle has completed its first pass on the waypoints, it will repeatedly go back to visit waypoints it missed the first time.

5.3 Requirements for this Exercise

Here's what we need to do:

- Augment the `pGenPath` application to keep track of which points a vehicle actually visits. The `pGenPath` app may be configured with a parameter `visit_radius=N`, setting the criteria for visiting a waypoint to be within N meters. The default should be 5. (Note the app will have to subscribe to `NAV_X` and `NAV_Y` if it isn't doing so already.)
- Configure your autonomy mission such that when it ends and returns to home, a helm behavior posts a message to `GENPATH_REGENERATE`.
- The `pGenPath` app registers for this message and regenerates a new path consisting of all the points missed on the previous pass.

- Once a new path has been regenerated, the vehicle resumes its mission until the new path has been traversed.
- The mission continues until there are no points left un-visited (as determined by the `pGenPath` application).
- For Assignment 2, demonstration will be on a single machine, all three vehicles simulated on the single machine.

5.4 Assignment 2 (check off) - Distributed TSP Two

You should demonstrate this re-planning capability from within a new mission folder created for this lab, in your `missions/lab_08` folder. As discussed above, it should be called `missions/lab_08/distributed_tsp_repla`. To make it easier to assess whether your planning is working, choose the `pPointAssign` option of assigning points grouped by East and West region so the paths are not overlapping one another. Your mission should look something like the video below:



Figure 4: In the Distributed TSP Two mission two vehicles are assigned randomly chosen points in the region based on the section of the region. Therefore their paths do not overlap and the mission is somewhat more efficient.

6 Distributed TSP Mission on Multiple Machines

In this last assignment, the goal is to convert your distributed TSP Replan mission to be launchable/runnable across separate machines. Again you will need a lab partner to demonstrate.

6.1 Preparing Your Mission for Launching on Multiple Machines

Your previous Distributed TSP mission will need to be modified a bit to support running on multiple machines. From Assignment 2 above, you should already have a `distributed_tsp_replan` mission folder, and your file structure should look like:

```
moos-ivp-extend/  
missions/  
  lab_08/  
    lab_08_baseline/           // Assignment 1 - check off  
    distributed_tsp_replan/     // Assignments 2,3 - check off
```

In Assignment 2 above, the mission folder for `distributed_tsp_replan` didn't change much in terms of the launch files and network configuration, compared to the structure from Lab 7. But in order for it to be run on multiple machines, it needs to borrow some of the structure from the `lab_08_baseline` mission folder. This is discussed next.

6.1.1 Preparing Your Mission Launch Scripts

The first thing you will need to do is to copy the `launch_vehicle.sh` and `launch_shoreside.sh` scripts from the `lab_08_baseline` mission into your `distributed_tsp_replan` mission folder.

```
$ cd moos-ivp-extend/missions/lab_08  
$ cp lab_08_baseline/launch_shoreside.sh distributed_tsp_replan/  
$ cp lab_08_baseline/launch_vehicle.sh distributed_tsp_replan/  
$ cp lab_08_baseline/launch.sh distributed_tsp_replan/
```

Note that in copying the `launch.sh` script into your `distributed_tsp_replan` folder that you will be overwriting the previous `launch.sh` script.

6.2 Assignment 3 (check off) - Distributed TSP Mission on Multiple Machines

To demonstrate this assignment:

- Enlist a lab partner to launch a Lab 8 shoreside community.
- The other lab partner should launch two distinct vehicles remotely on their machine.
- The shoreside lab partner hits the **Deploy** button.

Your mission should also re-demonstrate the re-planning aspect of Assignment 2.

If a student is working solo, the TA should be able pull the student's code from version control and launch the shoreside community. The TA will need to ensure that the student's version of `pPointAssign` is the version running on the shoreside, by setting the shell path order appropriately.

If the team consists of three people, one student should perform shoreside duties, while the other two students launch one vehicle each.

7 Instructions for File Organization of Assignments

7.1 Requested File Structure

This is a bit redundant by now, but here is the final requested file structure:

```
moos-ivp-extend/  
  missions/  
    lab_08/  
      lab_08_baseline/      // Assignment 1 - check off  
      distributed_tsp_replan/ // Assignment 2,3 - check off
```

7.2 Due Date

The due date for this lab is Thursday March 19th, 2026. We will start the next lab upon returning from Spring Break.